

SATELLITE IMAGE COMPARISON

Programming Technology Assignment

Name: Bertrand Billy Kombo

Date: 28 September 2026

<https://github.com/Billy237-Creator/satellite>

1. Objective

The goal of this assignment is to compare two satellite images represented by pixel value matrices. The program identifies the smallest bounding box that contains every difference between the images meaning that outside this area, both images are identical.

The Java implementation uses a single class built entirely with static methods. To keep the code clean and avoid redundancy, a single parameterized method calculates the four boundary coordinates (X1, X2, Y1, and Y2), while a shared helper method handles reading both the original and updated images.

2. Algorithm

The program starts by reading the dimensions N and M of the satellite images, then loads the pixel values of both the original and updated images into two separate matrices.

To pin down where the images differ, it looks for four key boundaries:

- **Y1** – the first row with a difference
- **Y2** – the last row with a difference
- **X1** – the first column with a difference
- **X2** – the last column with a difference

A single parameterized method, ***findBoundary()***, handles all four scans. Input parameters specify whether it scans rows or columns and whether it searches forward or backward.

If no differences are detected, the program simply reports that the images are identical. Otherwise, X1, X2, Y1, and Y2 define the smallest bounding box containing all changes.

3. Program structure

The program is implemented using a single Java class named **Main**. All methods are static, as required by the assignment. Each method has a specific responsibility in order to keep the program modular and avoid code duplication.

readImage()

Reads the pixel values of one satellite image and stores them in a two-dimensional integer array. The same method is called twice: once for the old image and once for the new image.

equalLine()

Compares either one row or one column of the two images. It returns true if all corresponding pixels are identical and false when a difference is found.

findBoundary()

Searches for one boundary of the differing area. The parameters determine whether the method searches rows or columns and whether it searches forward or backward. Therefore, the same method can determine X1, X2, Y1 and Y2.

findAllBoundaries()

Calls findBoundary() four times with different parameters to obtain all four coordinates.

printResult()

Displays either the four boundary coordinates or the message Images are identical.

solve()

Coordinates the complete process: reading the dimensions, reading both images, finding the boundaries and displaying the result.

main()

Starts the program by calling solve().

4. Java Source Code

The following Java source code implements the satellite image comparison algorithm. The program contains a single class and uses only static methods.

```
import java.util.Scanner;

public class Main {

    static int[][] readImage(Scanner sc, int n, int m) {

        int[][] img = new int[n][m];

        for (int i = 0; i < n; i++)

            for (int j = 0; j < m; j++) img[i][j] = sc.nextInt();

        return img;
    }

    static boolean equalLine(int[][] a, int[][] b, boolean rows, int k) {

        int len = rows ? a[0].length : a.length;

        for (int i = 0; i < len; i++) {

            int r = rows ? k : i, c = rows ? i : k;

            if (a[r][c] != b[r][c]) return false;
        }

        return true;
    }

    static int findBoundary(int[][] a, int[][] b, boolean rows, boolean reverse) {

        int limit = rows ? a.length : a[0].length;

        int k = reverse ? limit - 1 : 0, step = reverse ? -1 : 1;

        while (k >= 0 && k < limit) {

            if (!equalLine(a, b, rows, k)) return k;

            k += step;
        }

        return -1;
    }
}
```

```

    }

    static int[] findAllBoundaries(int[][] a, int[][] b) {

        return new int[]{findBoundary(a,b,true,false), findBoundary(a,b,true,true),
                           findBoundary(a,b,false,false),
                           findBoundary(a,b,false,true)};

    }

    static void printResult(int[] p) {

        if (p[0] == -1) System.out.println("Images are identical");

        else System.out.println("X1=" + p[2] + " X2=" + p[3] +
                                " Y1=" + p[0] + " Y2=" + p[1]);

    }

    static void solve() {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt(), m = sc.nextInt();

        int[][] oldImg = readImage(sc, n, m);

        int[][] newImg = readImage(sc, n, m);

        printResult(findAllBoundaries(oldImg, newImg));

        sc.close();

    }

    public static void main(String[] args) {

        solve();

    }

}

```

5. Tests

1. Test 1: Identical Images

This test verifies that the program correctly detects when the two satellite images are completely identical. **Input file:** test1.txt

```

3 3
0 0 0
0 5 0
0 0 0
0 0 0
0 5 0
0 0 0

```

```

C:\Users\Billy\Desktop\Satellite>javac Main.java
C:\Users\Billy\Desktop\Satellite>java Main < test1.txt
Images are identical

```

2. Single Different Pixel

This test verifies that the program correctly determines the boundaries when only one pixel is different between the two satellite images.

Input file: test2.txt

```
4 5
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 7 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

```
C:\Users\Billy\Desktop\Satellite>java Main < test2.txt
X1=3 X2=3 Y1=0 Y2=0
```

Expected result: X1=3 X2=3 Y1=0 Y2=0

Result: The program correctly identified the single different pixel at column 3 and row 0. Since there is only one different pixel, X1 equals X2 and Y1 equals Y2. Therefore, Test 2 passed successfully.

3. Test 3: Multiple Differences

This test verifies that the program correctly determines the smallest bounding rectangle when several pixels are different between the two satellite images.

Input file: test3.txt

```
4 5
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 9
0 0 0 0 0
0 8 0 0 0
0 0 7 0 0
```

```
C:\Users\Billy\Desktop\Satellite>java Main < test3.txt
X1=1 X2=4 Y1=0 Y2=3
```

Expected result: X1=1 X2=4 Y1=0 Y2=3

Result: The program correctly identified the boundaries of the area containing all differences. The first different column is X1=1, the last different column is X2=4, the first different row is Y1=0, and the last different row is Y2=3. Therefore, Test 3 passed successfully.

6. Conclusion

The satellite image comparison program was successfully implemented and tested. It determines the smallest rectangular area containing all differences between an old satellite image and a new satellite image.

The program follows the modularity requirements of the assignment by using a single Java class and only static methods. A common `readImage()` method is used to read both images, while the parametrized `findBoundary()` method is reused to determine `X1`, `X2`, `Y1` and `Y2`, avoiding unnecessary code duplication.

The three test cases demonstrate that the program correctly handles identical images, a single different pixel, and multiple differences. All tests produced the expected results.